

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

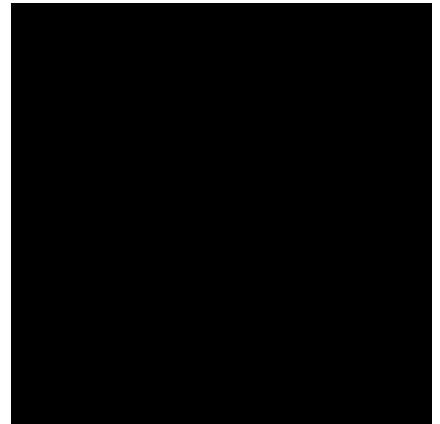
División de Ciencias de la Ingeniería

Área Profesional

Sistemas Operativos 2

Sección "A"

Ing. Otto Alejandro Soto Macal



"PRACTICA 2"

Melvin Eduardo Ordoñez Sapón RA | 202230552

Quetzaltenango, septiembre del 2026

"Id y enseñad a todos"

EJERCICIO 1:

```
Problems Output Debug Console Terminal Ports
melvin@mlvin:~/Escritorio/SOPES2/P2$ ./problema1
=====
SISTEMA DE PAGO EN LINEA
=====
Desea verificar el estado del canal? (s/n): s
Ingrese una palabra para verificar el canal: sistemas

[HIJO] Canal activo.
[HIJO] Palabra recibida: sistemas
[HIJO] Palabra invertida: sametsis

Ingrese numero de tarjeta (1000 - 9999): 2400

[HIJO] Procesando tarjeta 2400...

Resultado final: PAGO_APROBADO
melvin@mlvin:~/Escritorio/SOPES2/P2$
```

En esta ejecución se eligió realizar la verificación del canal. El padre envió la palabra sistemas al proceso hijo por medio de una tubería. El hijo recibió la palabra, la invirtió carácter por carácter y mostró sametsis, confirmando que la comunicación estaba funcionando.

Después se ingresó la tarjeta 2400. Como es un número par, el proceso hijo respondió con PAGO_APROBADO y el padre mostró el resultado final.

```
Problems Output Debug Console Terminal Ports
melvin@mlvin:~/Escritorio/SOPES2/P2$ ./problema1
=====
SISTEMA DE PAGO EN LINEA
=====
Desea verificar el estado del canal? (s/n): n

Verificacion omitida.

Ingrese numero de tarjeta (1000 - 9999): 1357

[HIJO] Procesando tarjeta 1357...

Resultado final: PAGO_RECHAZADO
melvin@mlvin:~/Escritorio/SOPES2/P2$
```

En esta ejecución se omitió la verificación del canal seleccionando la opción n. El programa continuó directamente con el ingreso del número de tarjeta. Se utilizó el número 1357, que es impar, por lo que el proceso hijo determinó que el resultado debía ser PAGO_RECHAZADO y envió esa respuesta al proceso padre.

1-¿Qué ocurriría si el hijo intentara leer de la tubería antes de que el padre haya escrito algo?

El proceso hijo quedaría bloqueado en la función read() esperando a que el padre escriba información en la tubería. Cuando existan datos disponibles, el hijo podrá continuar con su ejecución.

2- La segunda fase requiere que el hijo envíe una respuesta de vuelta al padre. ¿Por qué no es suficiente con la misma tubería usada en la verificación?

Porque en esta parte se necesita comunicación en ambos sentidos. El padre envía el número de tarjeta al hijo y el hijo debe devolver el resultado al padre. Por esta razón se utilizan dos tuberías, una para cada dirección.

3- ¿Qué problema concreto surgiría si ambos procesos intentaran leer y escribir sobre la misma tubería?

Podrían presentarse problemas de sincronización, ya que cualquiera de los procesos podría leer información que no estaba destinada para él. Esto dificultaría controlar correctamente el flujo de los datos y podría provocar comportamientos inesperados.

PROBLEMA 2:

```
melvin@mlvin:~/Escritorio/SOPES2/P2$ ./problema2
=====
REGISTRO DE PEDIDOS
=====
Ingrese unidades del pedido 1 (1 - 100): 10
Ingrese unidades del pedido 2 (1 - 100): 20
Ingrese unidades del pedido 3 (1 - 100): 30
Ingrese unidades del pedido 4 (1 - 100): 40
Ingrese unidades del pedido 5 (1 - 100): 50
Ingrese unidades del pedido 6 (1 - 100): 60
Ingrese unidades del pedido 7 (1 - 100): 70
Ingrese unidades del pedido 8 (1 - 100): 80
Ingrese unidades del pedido 9 (1 - 100): 90
Ingrese unidades del pedido 10 (1 - 100): 100
Ingrese unidades del pedido 11 (1 - 100): 15
Ingrese unidades del pedido 12 (1 - 100): 25
Ingrese unidades del pedido 13 (1 - 100): 35
Ingrese unidades del pedido 14 (1 - 100): 45
Ingrese unidades del pedido 15 (1 - 100): 55
Ingrese unidades del pedido 16 (1 - 100): 65
Ingrese unidades del pedido 17 (1 - 100): 75
Ingrese unidades del pedido 18 (1 - 100): 85
Ingrese unidades del pedido 19 (1 - 100): 95
Ingrese unidades del pedido 20 (1 - 100): 5

Todos los pedidos fueron registrados.
Enviando pedidos a las estaciones...

[ESTACION 1] Pedido recibido: 10 camisas
[ESTACION 2] Pedido recibido: 20 camisas
[ESTACION 1] Pedido recibido: 30 camisas
[ESTACION 2] Pedido recibido: 40 camisas
[ESTACION 1] Pedido recibido: 50 camisas
[ESTACION 2] Pedido recibido: 60 camisas
[ESTACION 2] Pedido recibido: 80 camisas
[ESTACION 1] Pedido recibido: 70 camisas
[ESTACION 2] Pedido recibido: 100 camisas
[ESTACION 1] Pedido recibido: 90 camisas
[ESTACION 1] Pedido recibido: 15 camisas
[ESTACION 2] Pedido recibido: 25 camisas
[ESTACION 1] Pedido recibido: 35 camisas
[ESTACION 2] Pedido recibido: 45 camisas
[ESTACION 1] Pedido recibido: 55 camisas
[ESTACION 2] Pedido recibido: 65 camisas
[ESTACION 1] Pedido recibido: 75 camisas
[ESTACION 2] Pedido recibido: 85 camisas
[ESTACION 2] Pedido recibido: 5 camisas
[ESTACION 1] Pedido recibido: 95 camisas

===== REPORTE ESTACION 2 =====
Pedidos procesados: 10
Total de camisas: 525

===== REPORTE ESTACION 1 =====
Pedidos procesados: 10
Total de camisas: 525

Todos los pedidos fueron procesados.
melvin@mlvin:~/Escritorio/SOPES2/P2$
```

En esta ejecución el proceso padre solicita el ingreso de 20 pedidos, donde cada valor representa la cantidad de camisas que contiene el pedido. Los valores se almacenan primero y, después de registrar el último, son enviados por medio de la tubería para que las dos estaciones comiencen a procesarlos.

Después de registrar los 20 pedidos, el padre los envía por una tubería compartida. Las dos estaciones, representadas por procesos hijo, van tomando los pedidos disponibles y acumulando la cantidad de pedidos y camisas que procesan.

En esta ejecución cada estación procesó 10 pedidos y acumuló 525 camisas. Entre ambas estaciones se procesaron los 20 pedidos y un total de 1050 camisas.

1-¿Es posible predecir cuántos pedidos procesará cada estación?

No se puede asegurar de antemano cuántos pedidos procesará cada estación, ya que los dos procesos compiten por leer el siguiente pedido disponible en la tubería. Esto depende de cuál proceso sea ejecutado primero por el sistema operativo en cada momento. Esta situación se relaciona con una race condition, porque el resultado depende del orden en que los procesos acceden al recurso compartido.

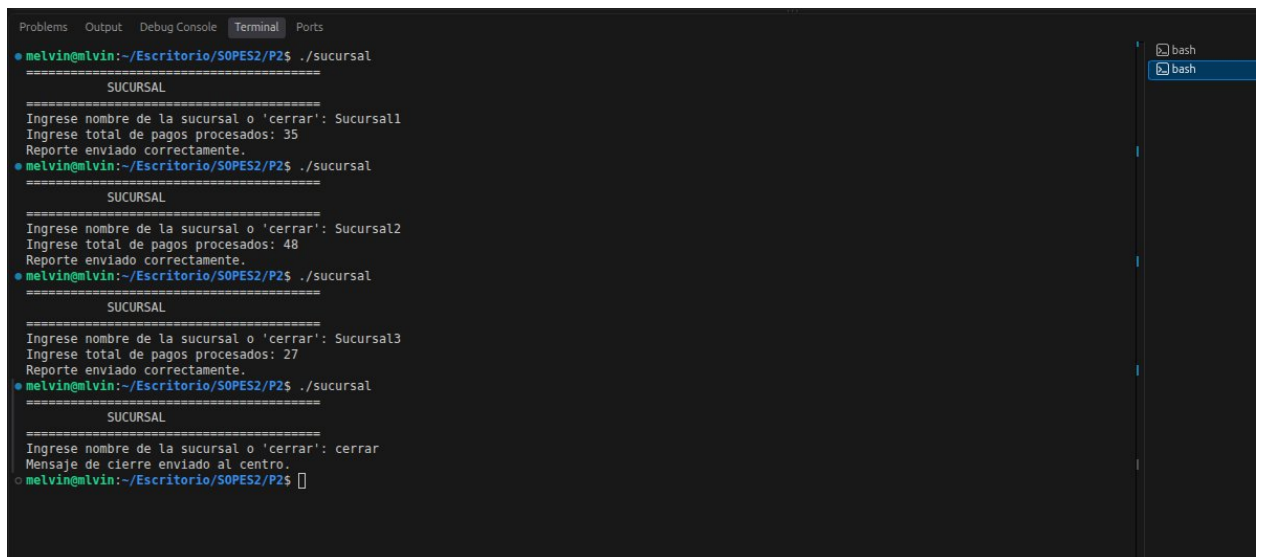
En nuestra ejecución coincidió que cada estación procesó 10 pedidos, pero esto no significa que siempre tenga que distribuirse de esa manera.

2-¿Qué pasaría si una estación no cerrara el extremo de escritura heredado?

Si una de las estaciones mantiene abierto el extremo de escritura de la tubería, los procesos lectores podrían quedarse esperando más información aunque el padre ya haya terminado de enviar todos los pedidos. Al permanecer un extremo de escritura abierto, no se detectaría correctamente el fin de la tubería y el programa podría quedar bloqueado.

EJERCICIO 3:

En este ejercicio se utilizaron dos programas independientes, centro.c y sucursal.c, que se comunican mediante una FIFO con nombre. Las sucursales envían su cantidad de pagos al centro y este los recibe, registra la hora de llegada y acumula el total. El proceso continúa hasta recibir el mensaje cerrar, momento en el que se muestra el resumen del día.



```
melvin@mlvin:~/Escritorio/SOPES2/P2$ ./sucursal
=====
SUCURSAL
=====
Ingrese nombre de la sucursal o 'cerrar': Sucursal1
Ingrese total de pagos procesados: 35
Reporte enviado correctamente.
melvin@mlvin:~/Escritorio/SOPES2/P2$ ./sucursal
=====
SUCURSAL
=====
Ingrese nombre de la sucursal o 'cerrar': Sucursal2
Ingrese total de pagos procesados: 48
Reporte enviado correctamente.
melvin@mlvin:~/Escritorio/SOPES2/P2$ ./sucursal
=====
SUCURSAL
=====
Ingrese nombre de la sucursal o 'cerrar': Sucursal3
Ingrese total de pagos procesados: 27
Reporte enviado correctamente.
melvin@mlvin:~/Escritorio/SOPES2/P2$ ./sucursal
=====
SUCURSAL
=====
Ingrese nombre de la sucursal o 'cerrar': cerrar
Mensaje de cierre enviado al centro.
melvin@mlvin:~/Escritorio/SOPES2/P2$
```

En esta ejecución se simulaban varias sucursales enviando sus reportes al centro de operaciones. Cada sucursal ingresó su nombre y la cantidad de pagos procesados. Después de enviar los tres reportes, se utilizó el mensaje cerrar para indicar al centro que debía finalizar la recepción de datos.



```
Problems Output Debug Console Terminal Ports
melvin@melvin:~/Escritorio/SOPES2/P2$ ./centro
=====
CENTRO DE OPERACIONES
=====
Esperando reportes de las sucursales...

Reporte recibido
Sucursal: Sucursal1
Pagos procesados: 35
Hora de llegada: 23:46:13

Reporte recibido
Sucursal: Sucursal2
Pagos procesados: 48
Hora de llegada: 23:46:45

Reporte recibido
Sucursal: Sucursal3
Pagos procesados: 27
Hora de llegada: 23:46:53

Mensaje 'cerrar' recibido.

=====
RESUMEN DEL DIA
=====
Reportes recibidos: 3
Total de pagos procesados: 110
melvin@melvin:~/Escritorio/SOPES2/P2$
```

El centro de operaciones permanece esperando los reportes enviados por las sucursales mediante la FIFO. Cada vez que recibe uno, muestra el nombre de la sucursal, la cantidad de pagos y la hora en que llegó. Al recibir el mensaje cerrar, el programa finaliza la recepción y muestra un resumen con 3 reportes recibidos y un total de 110 pagos procesados.

1-¿Por qué no fue necesario usar fork() para relacionar sucursal.c con centro.c?

No fue necesario utilizar fork() porque centro.c y sucursal.c son programas independientes. La FIFO tiene un nombre dentro del sistema, por lo que ambos programas pueden acceder a ella aunque no tengan una relación de padre e hijo. Uno puede abrirla para escribir y el otro para leer.

2-¿Qué ocurre si sucursal.c intenta abrir la FIFO antes de que centro.c la haya creado?

Si la FIFO todavía no existe, sucursal.c no podrá abrirla y se producirá un error. En nuestro programa se controla esta situación haciendo que la sucursal espere y vuelva a intentar la conexión hasta que el centro cree la FIFO.

[REPOSITORIO](#)